UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC
CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO – BCC

KAUAN HENRIQUE WERLICH

USO DE TÉCNICAS DE WEB SCRAPING PARA ALIMENTAÇÃO DE UMA ARQUITETURA RAG VOLTADA À RECUPERAÇÃO DE INFORMAÇÕES RELACIONADAS AOS CULTIVOS AGRÍCOLAS DE SC

JOINVILLE
2025

KAUAN HENRIQUE WERLICH

USO DE TÉCNICAS DE WEB SCRAPING PARA ALIMENTAÇÃO DE UMA ARQUITETURA RAG VOLTADA À RECUPERAÇÃO DE INFORMAÇÕES RELACIONADAS AOS CULTIVOS AGRÍCOLAS DE SC

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Fabiano Baldo

JOINVILLE
2025

KAUAN HENRIQUE WERLICH

USO DE TÉCNICAS DE WEB SCRAPING PARA ALIMENTAÇÃO DE UMA ARQUITETURA RAG VOLTADA À RECUPERAÇÃO DE INFORMAÇÕES RELACIONADAS AOS CULTIVOS AGRÍCOLAS DE SC

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Fabiano Baldo

BANCA EXAMINADORA:

Fabiano Baldo - Doutor
UDESC

Membros:

Rebeca Schroeder Freitas - Doutora
UDESC

Yuri Kaszubowski Lopes - Doutor
UDESC

Joinville, 16 de junho de 2025

RESUMO

Este trabalho propõe e avalia um pipeline para automatizar a extração, o pré-processamento, a vetorização e a indexação de conteúdos técnicos agrícolas disponíveis na plataforma EpagriTEC, visando alimentar um sistema de Retrieval-Augmented Generation (RAG). Na fase de coleta, empregou-se Web Scraping para navegar recursivamente pelas páginas de cultivos em Santa Catarina (ex.: banana), extraindo o HTML bruto. Em seguida, os documentos brutos foram limpos, removeu-se texto repetidoe e os caracteres foram normalizados. Nas etapas seguintes converteu-se os textos em *embeddings* e eles foram indexados localmente. Testes preliminares com consultas específicas (e.g., "sigatoka") revelaram cobertura de 100% e latência média de 12s. A análise qualitativa indica que o pipeline é viável, mas requer refinamento de *chunking* semântico e incorporação de descrições geradas para imagens. O trabalho futuro incluirá mais dados, métricas e avaliações mais avançadas, e o potencial uso de uma integração direta com um LLM para geração de respostas.

Palavras-chave: Web Scraping, Modelos de Linguagem em Larga Escala (LLMs), Geração Aumentada por Recuperação (RAG), Pré-processamento de Dados, Vetorização de Dados, Agricultura.

ABSTRACT

This work proposes and evaluates a pipeline to automate the extraction, preprocessing, vectorization, and indexing of technical agricultural content available on the EpagriTEC platform, aiming to feed a Retrieval-Augmented Generation (RAG) system. In the collection phase, web scraping was employed to recursively navigate the crop pages for Santa Catarina (e.g., banana), extracting raw HTML. Next, the raw documents were cleaned, duplicate text was removed, and characters were normalized. In subsequent steps, texts were converted into embeddings and locally indexed. Preliminary tests with specific queries (e.g., "sigatoka") revealed 100% coverage and an average latency of 12s. Qualitative analysis indicates that the pipeline is viable but requires semantic chunking refinement and incorporation of generated image descriptions. Future work will include additional data, metrics, more advanced evaluations, and the potential direct integration with an LLM for response generation.

Keywords: Web Scraping, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG); Data Preprocessing, Data Embedding, Agriculture.

LISTA DE ILUSTRAÇÕES

Figura 1 – Etapas de Web Scraping. Fonte: Shehu et al. (2024, p. 291). 14
Figura 2 – Pipeline proposto. Fonte: O autor. 26
Figura 3 – Distribuição do tamanho dos documentos utilizados. Fonte: O autor. 30
Figura 4 – Boxplot do tamanho dos documentos utilizados. Fonte: O autor. 30

LISTA DE TABELAS

Tabela 1 – Resumo dos Trabalhos Relacionados. 23
Tabela 2 – Cronograma de Tarefas – TCC2 (2025/2) 33

LISTA DE ABREVIATURAS E SIGLAS

FAISS	*Facebook AI Similarity Search*
LLM	Modelos de Linguagem em Larga Escala
NLP	Processamento de Linguagem Natural
PLM	Modelos de Linguagem Pré-Treinados
RAG	*Retrieval-Augmented Generation*
WS	*Web Scraping*

SUMÁRIO

1 INTRODUÇÃO . . . 9
1.1 OBJETIVO GERAL . . . 10
1.2 OBJETIVOS ESPECÍFICOS . . . 11
1.3 METODOLOGIA . . . 11
1.4 ESTRUTURA DO TRABALHO . . . 12
2 FUNDAMENTAÇÃO TEÓRICA . . . 13
2.1 MODELOS DE LINGUAGEM EM LARGA ESCALA . . . 13
2.2 EXTRAÇÃO DE DADOS VIA WEB SCRAPING . . . 13
2.3 PRÉ-PROCESSAMENTO DE TEXTO . . . 15
2.4 VETORIZAÇÃO E CONSTRUÇÃO DO BANCO VETORIAL . . . 16
2.4.1 Geração de *Embeddings* . . . 16
2.4.2 Armazenamento Vetorial . . . 16
2.5 PROCESSO DE RAG . . . 17
2.6 TRABALHOS RELACIONADOS . . . 18
2.6.1 Considerações sobre os Trabalhos Relacionados . . . 20
2.7 CONSIDERAÇÕES SOBRE O CAPÍTULO . . . 24
3 DESENVOLVIMENTO . . . 25
3.1 PROPOSTA . . . 25
3.2 DESENVOLVIMENTO DO PIPELINE . . . 25
3.2.1 Coleta de Dados por Web Scraping . . . 25
3.2.2 Pré-processamento dos Documentos . . . 26
3.2.3 Vetorização . . . 27
3.2.4 Indexação Vetorial . . . 27
3.3 CONSIDERAÇÕES SOBRE O CAPÍTULO . . . 27
4 ANÁLISE DOS RESULTADOS PARCIAIS . . . 28
4.1 METODOLOGIA DE AVALIAÇÃO . . . 28
4.2 RESULTADOS . . . 29
4.2.1 Análise Qualitativa dos Resultados . . . 29
4.3 CONSIDERAÇÕES SOBRE OS RESULTADOS . . . 30
5 CONSIDERAÇÕES PARCIAIS . . . 32
5.1 CRONOGRAMA DAS PRÓXIMAS ETAPAS . . . 32
REFERÊNCIAS . . . 34

1 INTRODUÇÃO

Segundo o Instituto Brasileiro de Geografia e Estatística (IBGE), o Produto Interno Bruto (PIB) do Brasil alcançou R$10,9 trilhões em 2023, um crescimento recorde em relação aos últimos anos (IBGE, 2024). O setor cujo desempenho apresentou maior destaque foi a agropecuária: com uma participação no PIB de R$677,6 bilhões, o resultado foi 15,1% maior do que no ano anterior. De acordo com o Ministério da Agricultura e Pecuária, o aumento significativo na produção de commodities impulsionou esse resultado positivo (BRASIL, 2024).

Uma das razões para esse sucesso é a crescente modernização da cadeia de produção agropecuária. A agropecuária é uma das áreas produtivas cuja eficiência mais aumentou graças à mecanização, inovação e informatização advindas das Revoluções Industriais e Digitais. Já no contexto atual, no qual é amplamente considerado que a Indústria 4.0 está revolucionando diversas áreas (KAGERMANN; WAHLSTER; HELBIG, 2013), os principais avanços ocorrem por meio de abordagens como a Agricultura de Precisão. A Agricultura de Precisão é uma abordagem de gestão agrícola que usa tecnologia da informação (TI) para otimizar o uso de recursos e reduzir impactos ambientais (DAS et al., 2018; OLIVEIRA; SILVA, 2023). Outras características importantes são a implementação de técnicas e tecnologias para melhorar a tomada de decisão, a automatização de tarefas e a obtenção de informações relevantes por meio da aquisição de dados. As áreas relacionadas, por se tratarem de terminologias recentes, por vezes são chamadas de Agricultura Digital ou Agricultura Inteligente dependendo do contexto (BASSOI et al., 2019).

Para implementar efetivamente essas estratégias, uma grande quantidade de informações técnicas é necessária aos profissionais da área, que envolvem especificações técnicas (como concentração de fertilizantes e água, distância entre sementes no plantio, detalhes sobre o solo, quantidade adequada de irrigação, etc.) diferentes para cada cultura a ser manejada. No Brasil, as informações de diversas culturas e regiões específicas já foram pesquisadas e catalogadas em livros e relatórios técnicos por empresas como a Empresa Brasileira de Pesquisa Agropecuária (EMBRAPA) ou a Empresa de Pesquisa Agropecuária e Extensão Rural de Santa Catarina (EPAGRI). Boa parte destas informações estão hoje disponíveis gratuitamente em diversos repositórios na Internet.

Contudo, a forma de busca por essas informações é predominantemente realizada por meio de motores de busca (do inglês, *search engines*) como Google, DuckDuckGo ou Bing. No entanto, a utilização dessas ferramentas apresenta certas limitações, pois a busca pode retornar documentos extensos que contêm apenas menções às palavras-chave utilizadas, bem como enfrentar dificuldades para mostrar resultados relevantes para itens com interpretações ambíguas. Os *search engines* mais avançados já implementam técnicas de Inteligência Artificial (IA) que tentam identificar os melhores resultados e decidir em qual ordem mostrá-los e também fazer pequenos resumos sobre o tema pesquisado. Entretanto, por utilizarem comumente conteúdos de propósito geral, pouco técnicos e detalhados, não apresentam resultados satisfatórios para

pesquisas com necessidades técnicas avançadas de respostas.

Paralelamente, nos últimos anos os Modelos de Linguagem em Larga Escala (LLMs - *Large Language Models*) de alta capacidade de geração de respostas se tornaram disponíveis e utilizados pela população em geral (como ChatGPT, Gemini, Llama). Modelos de Linguagem (*Language Models*) focam em compreender a distribuição da probabilidade de uma palavra em uma linguagem para que o modelo possa gerar um novo texto (ANNEPAKA; PAKRAY, 2025). Os LLMs se distinguem dos Modelos de Linguagem convencionais pela sua capacidade de realizar várias tarefas de Processamento de Linguagem Natural, incluindo gerar e classificar textos (ANNEPAKA; PAKRAY, 2025).

Como mencionado, a integração de LLMs com os *search engines* de diversos navegadores se tornou realidade. Essa inovação permitiu que esses LLMs fornecessem respostas rápidas baseadas no conteúdo em que foram treinados. As respostas geradas pelos LLMs geralmente apresentam boa acurácia para perguntas generalistas. Contudo, como esses LLMs são treinados somente com informações acessíveis na Web, a sua capacidade de responder corretamente para contextos específicos, como a agropecuária, é limitada.

Entretanto, empresas públicas de apoio agropecuário brasileiras oferecem uma ampla gama de conteúdos técnicos sobre o manejo de culturas, controle de pragas, doenças, plantas daninhas e gestão do solo, disponibilizados em seus blogs e intranets por meio de textos, imagens e vídeos. Logo, treinar um LLM com dados fornecidos por essas agências poderia auxiliar usuários na obtenção de respostas às suas dúvidas em relação a um tema agropecuário específico. Porém, isso só é possível se os dados estiverem disponíveis de forma que os LLMs possam consumi-los em seu treinamento.

Este trabalho tem a seguinte pergunta de pesquisa: Como extrair, formatar e estruturar dados disponíveis em blogs e intranets a fim de que sejam utilizados como fontes de informação em modelos de LLMs para o contexto agropecuário? O estudo assume que a extração é possível por meio de técnicas de Web Scraping (raspagem de dados) e posterior formatação utilizando um formato padrão de consumo desses modelos, sendo que o Web Scraping é o processo de extração de dados de páginas Web (LAWSON, 2015).

O trabalho está sendo viabilizado pela parceria do grupo Banco de Dados e Engenharia de Software (BDES-UDESC) e da Empresa de Pesquisa Agropecuária e Extensão Rural de Santa Catarina (EPAGRI), onde a EPAGRI disponibilizará dados públicos da plataforma de divulgação de conteúdos técnicos chamada EpagriTec [1].

1.1 OBJETIVO GERAL

O objetivo geral do trabalho é propor uma abordagem para extração, formatação e estruturação dos dados públicos disponibilizados pela EPAGRI no contexto da plataforma EpagriTec, para uso posterior por um modelo de LLM em respostas a perguntas sobre o contexto

[1] Disponível em: <https://sistemas.epagri.sc.gov.br/sedimob/consulta.action?subFuncao=epagriTec>

agropecuário.

1.2 OBJETIVOS ESPECÍFICOS

Os objetivos específicos são:

- Avaliar a literatura existente sobre técnicas de Web Scraping utilizadas na criação de *datasets*; técnicas de pré-processamento e enriquecimento de dados, técnicas de *Data Curation* (Curadoria de Dados) para análise de qualidade dos dados; e aplicação de *fine-tuning* em LLMs pré-treinados para refinamento. Com base nisso, decidir quais etapas serão necessárias à criação de um *dataset*.
- Investigar e definir as metodologias que serão utilizadas para o manuseio dos volumes de dados oriundos das técnicas de Web Scraping, e definir o formato do *dataset* que será construído a partir dos dados resultantes do Web Scraping.
- Investigar e definir um pipeline para extração e consumo de páginas Web da EPAGRI, de modo a obter o *dataset* necessário para o consumo por um LLM no contexto agropecuário, preservando o significado da hierarquia original.
- Elaborar um processo que, de acordo com as técnicas investigadas de Web Scraping e de Pré-Processamento, formate os dados obtidos anteriormente de maneira a prepará-los para uso posterior no treinamento de uma IA.

1.3 METODOLOGIA

Para fornecer uma visão geral das atividades e garantir que o projeto seja realizado de forma estruturada e eficiente, propõe-se o plano de trabalho que será detalhado nos parágrafos seguintes.

Inicialmente, proceder-se-á à revisão da literatura científica para determinar o estado atual do conhecimento acerca do assunto, considerando os trabalhos mais recentes e/ou similares. A pesquisa focará em tecnologias, arquiteturas e conceitos relevantes para o estudo, incluindo Web Scraping, *Data Curation* e LLMs, com ênfase em aplicações relacionadas à agricultura e agricultura de precisão.

Na sequência, será definida a melhor maneira para formatar e armazenar os dados. Em seguida, serão avaliadas as técnicas necessárias na automação do pipeline de extração de dados e atualização do LLM, considerando fatores como a estruturação e a hierarquia dos dados e das páginas Web do site alvo deste trabalho.

Por último, será realizada uma bateria de experimentações, com o intuito de validar a abordagem proposta no trabalho. Serão definidas as ferramentas a serem aplicadas, os protótipos a serem desenvolvidos, os conjuntos de dados avaliados, as metodologias experimentais, as métricas de avaliação e os resultados esperados.

1.4 ESTRUTURA DO TRABALHO

Este trabalho será dividido da seguinte maneira. No Capítulo 2 revisam-se os principais conceitos necessários à compreensão do trabalho e apresenta os trabalhos relacionados. O Capítulo 3 apresenta a proposta do *pipeline* de extração, pré-processamento e armazenamento. Já o Capítulo 4 apresenta uma análise dos resultados parciais. Finalmente, o Capítulo 5 apresenta as considerações parciais e os próximos passos.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo descrevem-se os conceitos fundamentais à compreensão deste trabalho. Serão explorados os conceitos relacionados à extração de dados de páginas Web e sua aplicação em LLMs. Além disso, será feita uma análise dos trabalhos relacionados ao tema.

2.1 MODELOS DE LINGUAGEM EM LARGA ESCALA

Modelo de Linguagem em Larga Escala (LLMs - *Large Language Models*) é um termo cunhado para se referir a Modelos de Linguagem Pré-Treinados (PLMs - *Pre-Trained Language Models* com grande quantidade de parâmetros (e.g., dezenas ou centenas de bilhões) (ZHAO et al., 2023). Por sua vez, o treinamento e a inferência de PLMs seguem o paradigma de pré-treinamento e ajuste fino, no qual modelos de linguagem com redes neurais recorrentes ou com Transformers são pré-treinados em grandes corpora de texto não rotulado da Web, para realizar tarefas como predição de palavras, e então são ajustados com pequenas quantidades de dados rotulados de tarefas específicas (MINAEE; MIKOLOV et al., 2024). Recentemente, os LLMs ganharam proeminência e destaque especialmente após o lançamento do GPT-3 (2020) e do ChatGPT (2022) pela empresa OpenAI.

2.2 EXTRAÇÃO DE DADOS VIA WEB SCRAPING

Raspagem da Web (WS - *Web Scraping*) é todo o processo de extrair dados de páginas Web (LAWSON, 2015). O *Web Scraping* automatiza a coleta de informações de sites.

WS era a única maneira de obter informações da Internet até o surgimento das APIs (*Application Programming Interface*), que são interfaces feitas para tornar a comunicação entre aplicações e servidores mais fácil (KHDER, 2021). Contudo, mesmo as páginas Web que oferecem APIs não são todas gratuitas e também são limitadas em relação a quantos e quais dados elas oferecem. Muitos desenvolvedores priorizam a manutenção do lado do cliente (*front-end*) e não dão a mesma atenção à API. Por isso, não se pode depender de APIs para adquirir dados da Internet, e é preciso aplicar estratégias de WS para consegui-los (MITCHELL, 2018; KHDER, 2021).

De acordo com Persson (2019) as etapas do WS podem ser divididas em 3 fases:

- **Fase de Busca**: Nesta etapa, o site desejado, que contém as informações ou dados relevantes, é acessado por meio do protocolo HTTP. Um programa de computador (*scraper*) – um software que simula a navegação humana para extrair dados de páginas web – envia requisições GET para o servidor e recebe uma resposta correspondente, que geralmente é um documento HTML. Ferramentas como cURL ou wget podem ser utilizadas para enviar essas requisições HTTP GET ao URL desejado, obtendo o documento HTML de volta na resposta. Isso simula a forma como os navegadores

acessam o conteúdo de páginas web. O *scraper* emula um usuário da web, navegando pelos sites.

- **Fase de Extração**: Após a obtenção do documento HTML, esta é a fase em que os dados de interesse são extraídos. As tecnologias comumente empregadas nesta fase incluem expressões regulares, bibliotecas de análise HTML (HTML *parsing libraries*) ou consultas XPath (XML Path) para localizar informações dentro do documento.

- **Fase de Transformação**: Finalmente, os dados de interesse que foram extraídos são transformados em uma versão estruturada. Essa transformação é feita para que os dados possam ser armazenados em formatos como CSV, SQL ou Excel.

A Figura 1 ilustra o fluxo dessas etapas.

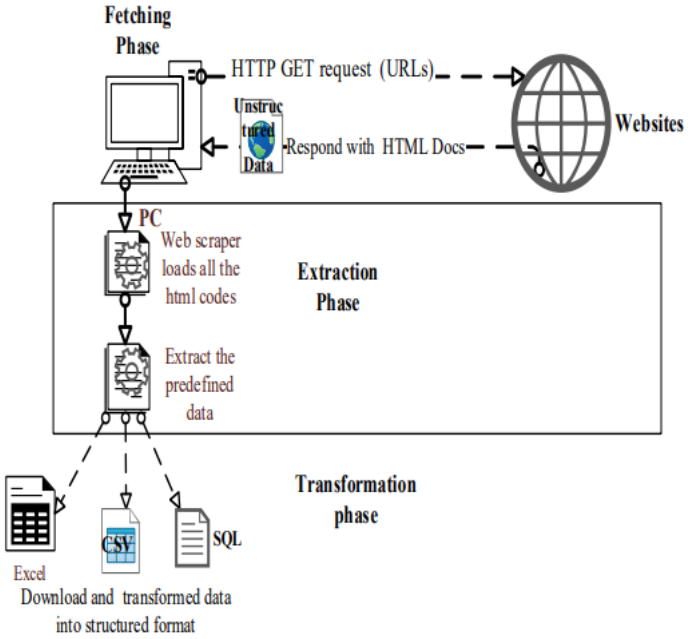

Figura 1 – Etapas de Web Scraping. Fonte: Shehu et al. (2024, p. 291).

Após a extração, também é comum aplicar procedimentos de limpeza e refinamento nos dados brutos, removendo ruído, espaços desnecessários, símbolos ou *tags* residuais. “Sem a limpeza adequada, transformando e estruturando os dados antes da análise, não se pode esperar encontrar padrões significativos” (RAMYA; SHREEDHARA; KAVITHA, 2011). A limpeza e o pré-processamento dos dados extraídos da web são etapas indispensáveis, que exigem a análise manual da estrutura original do conteúdo coletado, a fim de identificar padrões, remover ruídos e garantir a consistência dos dados para análise posterior (TARANNUM, 2019).

Além disso, a estrutura inconsistente é um dos maiores entraves a um processo de WS confiável. Segundo (WEERASINGHE, 2024), os métodos de WS tradicionais enfrentam desafios em lidar com *websites* dinâmicos, em superar medidas anti-scraping e em extrair dados estruturados de páginas Web não estruturadas.

Em resumo, o WS automatiza a coleta de dados estruturados ou semi-estruturados por meio de *scripts* capazes de acessar páginas Web, interpretar seu HTML e extrair os elementos desejados. Essa técnica se mostra essencial, especialmente em domínios onde APIs não estão disponíveis ou onde as informações estão dispersas em portais Web não estruturados.

2.3 PRÉ-PROCESSAMENTO DE TEXTO

O processo de pré-processamento de texto envolve a transformação de textos brutos em unidades que podem ser processadas por modelos computacionais, como redes neurais, e é frequentemente o primeiro módulo em um pipeline de Processamento de Linguagem Natural (NLP - *Natural Language Processing*) (CAMACHO-COLLADOS; PILEHVAR, 2018).

Segundo Elov, Khamroeva e Xusainova (2023), em um pipeline de NLP típico as etapas iniciais englobam coleta de dados e limpeza do texto. Nesse contexto, o pré-processamento de texto transforma dados brutos em unidades padronizadas para os modelos de linguagem. Upadhye (2020) reforça que identificar e eliminar ruídos (caracteres supérfluos) e padronizar formatos textuais são passos críticos, pois servem de precursor a tarefas como análise de sentimento, classificação de texto e recuperação de informação. No caso de textos extraídos da Web, isso inclui remover o chamado *boilerplate* (scripts, menus, anúncios etc.), que, segundo Fernández-Pichel et al. (2024), "empobrece o desempenho" de sistemas *downstream* devido à sua natureza repetida. Além disso, corrigem-se questões de codificação (UTF-8), removem-se duplicatas e normalizam-se convenções como pontuação, uso de maiúsculas/minúsculas e acentuação (para garantir consistência) (UPADHYE, 2020).

As etapas abaixo são recorrentes no pré-processamento de textos:

- **Remoção de ruído e sanitização**: eliminar elementos não informativos (ex.: tags HTML, *scripts*, anúncios, menus), pontuação irrelevante ou caracteres não-alfanuméricos (FERNÁNDEZ-PICHEL et al., 2024), e corrigir erros de codificação e de digitação. Por exemplo, Upadhye (2020) destaca que erros tipográficos, ortográficos e abreviações não padronizadas degradam a performance de tarefas de NLP, justificando a aplicação de corretores ortográficos e de normalização.

- **Normalização de texto**: padronizar o texto em caixa baixa (*lowercasing*), remover ou uniformizar acentos e normalizar formatos de data ou outros padrões; também costuma-se eliminar *stopwords* (palavras de baixa carga semântica), o que reduz ruído e a dimensionalidade dos dados (UPADHYE, 2020). Técnicas como lematização também podem ser aplicadas para agrupar formas variantes de palavras comuns.

- **Remoção de duplicatas**: eliminar repetições exatas ou quase-exatas de trechos textuais evita vieses e redundância no corpus final.

- **Estruturação de registros**: organizar cada segmento de texto limpo como um registro padronizado (ex.: chunks ou documentos) acompanhado de metadados (origem, data,

tópicos). (MURTAZA et al., 2025) ilustram essa etapa convertendo dados tabulares em strings JSON agrupadas por conceito e dividindo documentos de políticas em chunks vetorizáveis. Esse processo garante um corpus coerente e indexável para recuperação de informação subsequente.

Seguindo essas etapas, produz-se um corpus confiável e bem estruturado, pronto para indexação e uso em pipelines RAG.

2.4 VETORIZAÇÃO E CONSTRUÇÃO DO BANCO VETORIAL

2.4.1 Geração de *Embeddings*

Na etapa de vetorização, cada documento ou fragmento textual é convertido em um vetor numérico de dimensão fixa, conhecido como *embedding*. Modelos pré-treinados baseados em *transformers* permitem extrair *embeddings* densos que capturam significado dos textos. Por exemplo, Sentence-BERT (SBERT) altera o BERT original com uma arquitetura siamesa para produzir vetores de sentença comparáveis via similaridade de cosseno (REIMERS; GUREVYCH, 2019). SBERT reduz drasticamente o custo de busca semântica: o tempo de encontrar o par mais similar entre 10 mil sentenças cai de 65 horas (com BERT completo) para 5 segundos, mantendo a mesma acurácia (REIMERS; GUREVYCH, 2019).

Em suma, esses *embeddings* densos permitem representar sentenças ou documentos por vetores de dimensão fixa que podem ser indexados e posicionados a textos semanticamente semelhantes próximos uns dos outros em um espaço vetorial. Com isso, um trecho de consulta (e.g., pergunta do usuário) pode ser comparado via similaridade de cosseno aos vetores dos documentos, identificando os textos mais relevantes.

2.4.2 Armazenamento Vetorial

Após gerar os *embeddings*, eles são indexados em um banco de dados vetorial, projetado para buscas eficientes por similaridade em espaços de alta dimensão. Por exemplo, no estudo de Akbar (2025), utilizou-se a biblioteca FAISS (Facebook AI Similarity Search) para indexar os *embeddings* e acelerar a recuperação dos documentos mais relevantes.

Bancos de dados vetoriais armazenam grandes coleções de vetores e oferecem busca de vizinhança aproximada (ANN – *Approximate Nearest Neighbor*). Técnicas comuns de indexação incluem *hashing* (LSH), quantização (como Product Quantization/IVFADC), estruturas em árvore (como FLANN e Annoy) e grafos de vizinhança como o *Hierarchical Navigable Small World* (HNSW) (PAN; WANG; LI, 2024). Essas técnicas equilibram desempenho e precisão de busca, permitindo que milhões ou bilhões de vetores sejam consultados com latência reduzida. Segundo Pan, Wang e Li (2024), abordagens baseadas em grafos, como HNSW, fornecem alta precisão prática, enquanto métodos de quantização são úteis para escalabilidade e atualização dinâmica.

Diversos *frameworks* implementam essas abordagens, como FAISS e Milvus, além de extensões vetoriais em motores como Lucene ou Elasticsearch. Esses sistemas permitem recuperar, com alta eficiência, os k-vizinhos mais próximos de um vetor de consulta com base em métricas como distância Euclidiana ou produto interno.

Essa infraestrutura é essencial na arquitetura RAG: quando o LLM recebe uma pergunta, ela é convertida em vetor e comparada aos vetores armazenados no banco vetorial. As passagens mais semelhantes são então recuperadas e incorporadas ao contexto da resposta, permitindo que o LLM produza respostas fundamentadas em informações relevantes e atualizadas do domínio.

2.5 PROCESSO DE RAG

A técnica de Retrieval-Augmented Generation (RAG) consiste em recuperar pedaços (*chunks*) de documentos relevantes de uma base de conhecimento externa, por meio de cálculo semântico de similaridade (GAO et al., 2023). Em RAG, o LLM é complementado por um índice vetorial denso de documentos externos. Esse arranjo permite que informações atualizadas de fontes externas sejam acessadas no momento da geração do texto, aumentando a precisão e a credibilidade das respostas em tarefas intensivas em conhecimento. Gao et al. (2023) ressaltam que o RAG incorpora dados de bases de conhecimento especializadas, elevando a exatidão das respostas e permitindo atualizações contínuas de domínio específico (GAO et al., 2023). Em outras palavras, o RAG mitiga limitações dos modelos puramente paramétricos (como as "alucinações") ao adicionar uma camada de busca documental, tornando o modelo adaptável a novos conhecimentos.

Além disso, embora dados estruturados possam fornecer respostas muito precisas (devido à clareza de seus metadados), sua integração em RAG envolve trade-offs entre custo de ingestão e ganho de desempenho em produção (MURTAZA et al., 2025). Dados não estruturados podem ser indexados diretamente em um banco vetorial, e basta buscar os trechos relevantes e fornecê-los ao LLM, o que reduz o número de chamadas ao modelo e mantém a latência baixa. Já dados estruturados (tabelas, registros de banco de dados) não se encaixam tão bem nesse fluxo. Ou gera-se dinamicamente uma consulta (por exemplo, SQL) — o que aumenta chamadas ao LLM, custo e latência — ou converte-se o próprio registro em texto/*embedding* antes da inferência, o que eleva o trabalho inicial de pré-processamento.

A arquitetura típica de RAG envolve vários componentes integrados. A fonte de dados é a base de conhecimento que alimenta o sistema. Os dados extraídos passam por pré-processamento (Seção 2.3) antes de serem vetorizados e armazenados em um banco vetorial (Seção 2.4). Um LLM então pode ser conectado a esse banco: ao receber uma consulta, ele recupera os vetores mais semelhantes ao contexto (via busca vetorial) e os incorpora ao prompt do modelo, produzindo respostas informadas e baseadas em evidências. Esse fluxo – fonte de dados, extração, limpeza, vetorização, armazenamento e consulta – define as principais etapas da solução RAG.

2.6 TRABALHOS RELACIONADOS

Os trabalhos relacionados foram selecionados com foco em aplicações que envolvem técnicas de Web Scraping, vetorização de documentos e uso de arquiteturas RAG para recuperação inteligente de informações. Esta seção apresenta um resumo da revisão da literatura realizada.

Rezayi et al. (2022) desenvolveram o AgriBERT, pré-treinado em uma base de dados textuais de alimentos e agricultura, e o afinam para "*matching*" semântico entre descrições de alimentos e dados nutricionais. O *fine-tuning* desse modelo incluiu o uso de conhecimento externo (a ontologia FoodOn), o que resultou em desempenho superior na tarefa de correspondência alimentar-nutrição.

Em paralelo, Jiang et al. (2025) propuseram o KALLM, um LLM agrícola orientado por conhecimento, onde prepararam um extenso dataset (aproximadamente 220 mil pares de perguntas e respostas) a partir de documentos agrícolas. Eles aplicaram um *fine-tuning* coordenado que amplifica *tokens* específicos do domínio agrícola e introduziram um mecanismo RAG auto reflexivo baseado em correspondência de tópicos para aprimorar a recuperação de evidências. Esse pipeline (*Fine-tuning* Supervisionado (SFT - *Supervised Fine-tuning* coordenado + RAG) superou significativamente LLMs de domínio geral em fluência e precisão no contexto agroindustrial.

Panoutsopoulos et al. (2024) também investigaram o fine-tuning no domínio agrícola, comparando BERTs gerais e específicos (Agriculture-BERT, SciBERT etc.) para extração de termos técnicos. Eles mostraram que o Agriculture-BERT geralmente supera os demais e que a estratégia de congelar ou atualizar camadas específicas durante o treino influencia fortemente a capacidade de identificar termos já vistos, sinônimos ou termos novos.

Para sistemas de pergunta-resposta (QA) e chatbots agrícolas, técnicas de Recuperação-Aumentada por Geração (RAG) têm sido adotadas com destaque. Akbar (2025) avalia empiricamente o efeito de estratégias de re-ranking e computação em memória em modelos RAG para QA agrícola de pequena escala. Seu pipeline usa Sentence Transformer para *embeddings*, FAISS para busca vetorial e um LLM pré-treinado para gerar respostas. Observou-se que o re-ranking baseado em classificação zero-shot trouxe ganhos marginais, enquanto o uso de FAISS reduziu drasticamente a latência de recuperação, melhorando a adequação do sistema a cenários em tempo real.

Zhang et al. (2025) propuseram o BeefBot, um *chatbot* para pecuaristas que compara diretamente duas estratégias de preparação de um LLM *open-source*: (i) *fine-tuning* de um modelo base (LLaMA 2 13B) com dados agropecuários, e (ii) um *pipeline* RAG. Utilizando um corpus pré-indexado de artigos e relatórios sobre pecuária, aplicaram pré-processamento (remoção de *boilerplate*, tokenização e *chunking* de aproximadamente 500 *tokens*), geraram *embeddings* com Sentence-Transformer (all-MiniLM) e indexaram-nos em FAISS. Na abordagem RAG, consultam o índice para recuperar trechos relevantes que são então fornecidos ao LLaMA2 (afinando-o com LoRA) para geração; na abordagem de *fine-tuning*, ajustam o LLaMA2 diretamente no

corpus. Os testes mostraram que o pipeline RAG alcançou 85% de precisão em Q&A, contra 78% do fine-tuning puro, além de reduzir em 30% a latência de resposta. Esses resultados indicam que RAG pode superar *fine-tuning* direto em cenários de perguntas agrícolas, oferecendo alta precisão sem depender de grandes volumes de dados anotados.

Glukhikh, Chernysheva e Shentsov (2024) apresentam um sistema de suporte à decisão para estufas inteligentes baseado em LLM com Recuperação Aumentada por Geração (RAG). A arquitetura proposta integra três bases de conhecimento — observações de sensores (um banco estruturado com leituras de temperatura, umidade e detecção de doenças), uma base de doenças e medicamentos e uma base de referência geral — que são pré-processadas, divididas em *chunks* e vetorizadas por um modelo *Transformer*. Para a recuperação, empregam um esquema híbrido de busca que combina similaridade vetorial e BM25, permitindo selecionar os trechos mais relevantes conforme a consulta do usuário. O módulo RAG injeta esses fragmentos no *prompt* de um agente controlador (implementado sobre Llama2 7B LoRA), que orquestra as ferramentas de busca e gera respostas fundamentadas. Nos experimentos com dados reais de estufa, o sistema demonstrou ser capaz de diagnosticar doenças de plantas e sugerir tratamentos de forma confiável, reduzindo o risco de "alucinações" do modelo ao confirmar quando não dispõem de informação suficiente no contexto.

Ainda no domínio da agricultura digital, Xiong et al. (2025) criaram uma base de conhecimento local em proteção vegetal: vetorizam 7000 artigos científicos e selecionam 568 para gerar dados Q&A (perguntas e respostas) via LLM, resultando em um conjunto de aproximadamente 9000 pares de perguntas e respostas limpos. Em seguida aplicam LoRA para ajustar um modelo InterLM-20B, gerando o InterLM-20B-LoRA integrado ao LangChain e à base vetorial. Crucialmente, incorporam RAG para permitir que o modelo recupere conhecimento específico de campo antes de responder, mitigando alucinações. O sistema resultante atingiu 97% de acurácia nas respostas, evidenciando que a combinação de *fine-tuning* eficiente (LoRA) com pipeline de recuperação melhora significativamente a precisão em QA agrícola especializado. Em outras palavras, pipelines que incluem *embeddings* de documentos de domínio, mecanismos de recuperação e integração com LLMs conseguem respostas mais fidedignas.

Outros trabalhos reforçam a importância de arquiteturas RAG bem arquitetadas em diferentes domínios. Por exemplo, Vaghefi et al. (2023) apresentam o ChatClimate, um protótipo de AI conversacional para questões climáticas. Eles conectam o GPT-4 ao relatório AR6 do IPCC (fonte autoritativa de clima) por meio de um pipeline de LangChain: o texto foi dividido em trechos, embutido via `text-embedding-ada-002` e indexado para recuperação. Comparando três configurações (GPT-4 puro, GPT-4 consultando somente AR6, e uma versão híbrida GPT-4 + AR6), verificaram que o modelo híbrido gerou respostas com mais acurácia, demonstrando que ancorar o LLM em literatura científica relevante via RAG aumenta a confiabilidade do sistema.

Sohn et al. (2025) introduziram o RAG^2 (Rationale-Guided RAG) para Q&A médico. Eles combinam três inovações: filtragem de trechos informativos com base em "perplexidade de razão", geração de raciocínios pelo LLM para guiar as consultas, e recuperação balanceada

de quatro corpora biomédicos para mitigar vieses. Esse pipeline estruturado de RAG superou o estado da arte em benchmarks médicos, alcançando ganhos de até 6,1% em acurácia.

Já no campo jurídico, Ma et al. (2025) criam o SDD-LawLLM: geram sinteticamente dados estilo "Chain-of-Thought" a partir de normas via GPT-4, depois realizam *fine-tuning* em um LLM jurídico e empregam RAG para incorporar informação legal nos contextos. O resultado foi um modelo que supera LLMs genéricos em tarefas do benchmark LawBench, atribuindo parte dos avanços ao uso de RAG para contextualizar respostas jurídicas.

Em síntese, os trabalhos revisados enfatizam um pipeline comum: (i) coleta ou uso de dados especializados (literatura, relatórios ou documentos do domínio); (ii) pré-processamento desses textos; (iii) vetorização (embeddings); (iv) indexação (por exemplo, FAISS ou outros bancos vetoriais) para recuperação eficiente; (v) mecanismos de recuperação (ex: similaridade vetorial, re-ranking) que selecionam trechos relevantes; e (vi) integração com o LLM para geração da resposta. Poucos detalham o uso de WS, observa-se que muitos assumem acesso a bases de dados especializadas. Yang et al. (2024) e Samuel et al. (2025) são trabalhos ainda não publicados ("pré-prints")que também fazem uso de um RAG para complementar as respostas de um LLM, mas também não usam WS para obter seus dados. O uso de WS automatizado seria natural para compilar conteúdos atualizados (como notícias agrícolas, boletins técnicos e publicações oficiais) que alimentam essas bases. Em particular, a incorporação de pipelines de RAG tem se mostrado eficaz para conter alucinações do modelo e garantir precisão nas respostas, seja no contexto agrícola ou em outras áreas de aplicação.

2.6.1 Considerações sobre os Trabalhos Relacionados

Com base nos trabalhos levantados na Seção 2.6, a Tabela 1 compara os trabalhos com base nos seguintes critérios de análise:

- **Técnica Principal**: verifica como cada estudo aborda a integração entre recuperação de documentos e geração de texto, permitindo identificar quais abordagens priorizam atualização dinâmica vs. especialização por *fine-tuning*.

- **Fonte Principal de Dados**: evidencia a natureza e a heterogeneidade do corpus utilizado — textos agrícolas, relatórios técnicos, mídias multimodais — apontando o grau de independência em relação a bases estáticas ou a coleta automatizada.

- **Pré-Processamento**: expõe as práticas de limpeza e *chunking* aplicadas antes da vetorização, fator determinante para a qualidade dos *embeddings* e a eficiência da busca semântica.

- **Ferramenta de *Embedding***: reflete o método de conversão texto para vetor, apontando *trade-offs* entre modelo leve versus representação de domínio mais precisa.

A análise dos resultados contidos na Tabela 1 revela alguns padrões importantes. Primeiro, percebe-se que na maioria dos trabalhos com RAG utiliza-se *embeddings* gerados por ferramentas comerciais como o ChatGPT e APIs da OpenAI, que, embora pagas (i.e., ferramentas de propriedade intelectual de alguma empresa, não disponíveis ao público de forma completamente gratuita), oferecem atualmente as representações semânticas mais precisas e avançadas do mercado. Mas isso compromete a reprodutibilidade, a acessibilidade, a transparência e os custos de produção ao adotá-las como base de seus estudos. Dentre as ferramentas listadas nesses trabalhos, as principais alternativas *open-source* e/ou gratuitas foram o Sentence-Transformers (*embedder*), o FAISS (buscas em RAG) e o LLaMA (LLM). Além disso, observa-se variedade de *embedders*, e ausência das etapas de WS.

Quanto ao Pré-Processamento, a maioria dos autores enfatiza a remoção de partes repetidas e o *chunking* em fragmentos de tamanho controlado, prática que se mostra essencial para otimizar tanto a indexação quanto a coerência do contexto recuperado. Por fim, a Fonte de Dados distingue-se pela crescente adoção de pipelines que incorporam conteúdo multimodal ou bases especializadas (em domínios específicos, como medicina, agricultura ou direito), enquanto projetos mais antigos ou genéricos podem apoiar-se em corpora estáticos ou APIs de terceiros.

Também percebe-se que o RAG oferece excelentes resultados para a recuperação e geração de informações relevantes. Usar dados provenientes de WS também oferece potencial se comparado ao uso de corpus fixos: Ao extrair conteúdo diretamente de sites e portais agrícolas, o sistema mantém a base de conhecimento sempre atualizada (novas técnicas de cultivo, alertas fitossanitários, boletins climáticos), o que reduz significativamente o risco de respostas desatualizadas e melhora a relevância das recomendações. O módulo de WS também pode ser adaptado para incluir novas fontes, sem alterar o restante do pipeline de indexação e geração.

De forma complementar, observa-se na Tabela 1 que, os 10 trabalhos revisados empregam diferentes estratégias de recuperação de informações:

- 2 empregam apenas *fine-tuning* de LLMs (Rezayi et al. (2022) e Panoutsopoulos et al. (2024)).
- 3 combinam *fine-tuning* + RAG (Jiang et al. (2025), Xiong et al. (2025) e (MA et al., 2025)).
- 5 adotam exclusivamente RAG ou variantes (Zhang et al. (2025), Akbar (2025), Glukhikh, Chernysheva e Shentsov (2024), Vaghefi et al. (2023) e Sohn et al. (2025)).

Esse predomínio de RAG em relação ao *fine-tuning* puro reflete a capacidade do RAG de permitir incorporar informações recentes sem retrainar completamente o modelo, bastando atualizar a base vetorial. Já o *fine-tuning* exige conjuntos de dados grandes e estáticos, além de custosas etapas de re-treinamento para cada novo domínio ou atualização de conteúdo. *Fine-tuning* também requer dados anotados em larga escala (Q&A, relatórios), o que nem sempre

está disponível. RAG prescinde de anotações, trabalhando com texto bruto indexado, reduzindo o custo de criação de corpora específicos.

Por esses motivos, os trabalhos recentes privilegiam RAG, ou o combinam com *fine-tuning* seletivo, buscando um equilíbrio entre especialização do modelo e agilidade na incorporação de novos conhecimentos.

Portanto, tendo em vista os pontos levantados, as principais técnicas utilizadas ao longo desse trabalho serão RAG e WS. Por mais que outros trabalhos (notavelmente Zhang et al. (2025)) tenham um escopo parecido ao que será proposto, nenhum deles apresenta WS como etapa de obtenção de dados. Além disso, nenhum deles é um trabalho nacional, e as soluções adotadas não refletem a realidade dos dados de treinamento disponíveis na EPAGRI, o que reforça a necessidade de uma investigação na integração entre WS e RAG.

Tabela 1 – Resumo dos Trabalhos Relacionados.

Trabalho	**Técnica Principal**	**Fonte Principal de Dados**	**Pré-Processamento**	**Ferramenta de Embedding**
Rezayi et al. (2022)	Pré-treino + Fine-tuning	Corpora de texto alimentares; ontologia FoodOn	Limpeza de HTML, remoção de scripts, tokenização	BERT-based embeddings
Vaghefi et al. (2023)	RAG (GPT-4 + AR6 IPCC)	Relatórios IPCC AR6	Chunking em trechos de 1000 tokens, remoção de boilerplate	text-embedding-ada-002
Glukhikh, Chernysheva e Shentsov (2024)	RAG	Leituras de sensores; base de doenças; base referência	Limpeza de dados heterogêneos, chunking	Transformer-based embeddings
Panoutsopoulos et al. (2024)	Fine-tuning de BERT	Corpus anotado de textos agrícolas	Tokenização, lematização, remoção de stopwords	BERT embeddings
Zhang et al. (2025)	RAG / Fine-tuning	Corpus pré-indexado (papers, relatórios)	Limpeza de boilerplate, chunking	SentenceTransformer (all-MiniLM)
Jiang et al. (2025)	Fine-tuning coordenado + RAG	220000 pares Q&A de documentos agrícolas	Filtragem temática, limpeza de ruído	text-embedding-ada-002
Akbar (2025)	RAG + Re-ranking	Conjunto QA agrícola anotado	Normalização, stemming, remoção de duplicatas	SentenceTransformer (all-MiniLM)
Xiong et al. (2025)	LoRA + RAG (QA)	Base vetorizada de artigos de proteção vegetal	Extração de seções, limpeza, chunking	OpenAI/Vicuna embeddings
Sohn et al. (2025)	RAG² (com raciocínio)	4 corpora biomédicos	Filtragem por perplexidade, remoção de ruído	BERT-based embeddings
Ma et al. (2025)	Fine-tuning sintético + RAG	Dados legais sintéticos (CoT)	Geração de Chain-of-Thought, chunking	text-embedding-ada-002
Método Proposto	RAG + Web Scraping	Conteúdo web agrícola (sites, manuais, notícias)	Parsing HTML, remoção de padrões, chunking	OpenAIEmbeddings

Fonte: O autor.

2.7 CONSIDERAÇÕES SOBRE O CAPÍTULO

Este capítulo estabeleceu os fundamentos técnicos do pipeline RAG, desde a extração de dados via WS até a geração de respostas fundamentadas por LLMs. As seções de pré-processamento, vetorização e processo de RAG forneceram o arcabouço conceitual necessário para compreender cada etapa da proposta. Na revisão das soluções correlatas, identifica-se que, embora haja maturidade em técnicas de RAG e *fine-tuning*, ainda existe espaço para aprofundar o uso de WS como fonte primária, especialmente em contextos regionais como Santa Catarina. Essas observações guiarão a definição do *pipeline* deste trabalho.

3 DESENVOLVIMENTO

Este capítulo apresenta a proposta de um pipeline de coleta de dados, pré-processamento, vetorização e armazenamento vetorial dos dados da plataforma EpagriTEC para uso futuro na geração de respostas a um usuário. A seção 3.1 apresenta o detalhamento da abordagem de solução proposta, e a Seção 3.2 apresenta a implementação parcial do modelo proposto.

3.1 PROPOSTA

Na medida em que a necessidade de recuperação rápida e eficiente de informações em vários setores tem impulsionado o desenvolvimento de LLMs e *chatbots* especializados em diversos domínios, essa demanda também se faz presente no setor agropecuário.

O EpagriTec é uma plataforma da Epagri voltada a técnicos agrícolas de Santa Catarina. Ela organiza o conhecimento por cadeias produtivas importantes (como tomate, cebola, arroz irrigado), oferecendo para cada cultura tópicos técnicos sobre manejo, nutrição, fitossanidade, etc. Cada cadeia é subdividida em temas específicos (e.g., adubação, cultivares, controle de pragas), permitindo que o usuário navegue de acordo com sua necessidade. Os conteúdos podem ser acessados por página web. Em suma, o portal apresenta informações agrícolas organizadas por cultura e manejo.

Informações como as disponíveis na plataforma EpagriTEC são de grande valor para os profissionais da área, mas a página Web é antiga e apresenta baixa usabilidade, se comparada ao padrão atual de Desenvolvimento Web. Logo, um LLM que consiga usar as informações para responder às demandas de um usuário da plataforma tem grande potencial.

Portanto, com base na revisão dos trabalhos relacionados apresentada na Seção 2.6, desenvolve-se a proposta do uso de técnicas de *Web Scraping* e RAG para indexar essas informações num banco de dados vetorial, pronto para o acesso de um LLM, através de um *pipeline* com as etapas iniciais frequentemente mencionadas na literatura, adaptadas para este trabalho.

3.2 DESENVOLVIMENTO DO PIPELINE

O *pipeline* a ser desenvolvido, detalhado na Figura 2, terá quatro etapas principais. (i) coleta de dados especializados (através de técnicas de WS); (ii) pré-processamento dos documentos (remoção de padrões, conteúdo irrelevante e limpeza do texto); (iii) vetorização e (iv) indexação. Essas etapas são detalhadas nas Seções subsequentes.

3.2.1 Coleta de Dados por Web Scraping

A etapa inicial se caracteriza pela obtenção dos dados brutos da plataforma EpagriTEC. Os dados são extraídos por meio de técnicas de WS, de acordo com os seguintes passos:

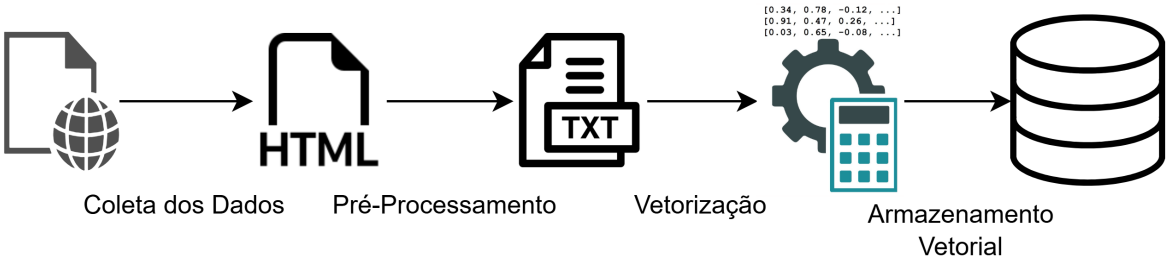

Figura 2 – Pipeline proposto. Fonte: O autor.

- Primeiramente o *scraper* acessa o link base da página, i.e, a URL base do *website*, que contém o menu.
- Em seguida o *scraper* procura todos os botões na página com identificador "mais", para expandir os textos minimizados.
- Então o *scraper* obtém uma cópia local do HTML da página, a qual ele armazena em uma pasta de destino.
- Na sequência, o *scraper* extrai via JavaScript uma lista de sub-códigos `dadosTabela`. Esses códigos estão presentes em todas as URLs do site e são identificadores únicos das páginas.
- Para cada sub-código existente na página, o *scraper* verifica se ele já foi processado anteriormente. Se não, repetem-se os passos anteriores, de maneira recursiva. Se já foi processado, prossegue-se ao próximo da lista.

Esse processo garante a coleta de todas as páginas vinculadas hierarquicamente a partir do código (URL) inicial.

3.2.2 Pré-processamento dos Documentos

Cada arquivo HTML coletado é então convertido em um arquivo de texto bruto (.txt), mantendo a hierarquia original (título, subtítulos e parágrafos). A proposta original do trabalho pretendia convertê-los em arquivos JSON, mantendo a estrutura e a organização das páginas originais. No entanto, após a coleta dos dados verificou-se que a organização das informações nas diferentes páginas do portal não seguia um único padrão, e manter essas informações trazia pouco valor para o trabalho e aumentava o ruído já mencionado na literatura ao usar informações estruturadas em um processo de RAG. Dessa forma, optou-se por remover essas informações e deixar o texto o mais limpo possível - lembrando a representação visual e sequencial que um usuário teria ao lê-lo na própria página.

Aplicou-se a sanitização de caracteres, envolvendo: a remoção de todos os caracteres não textuais, normalização das codificações de caracteres, remoção de *tags* e *scripts*. Devido

à heterogeneidade das páginas, optou-se por tratar cada documento HTML como uma única "unidade", sem fragmentação automática por seções. Essa estratégia simplifica o manejo inicial, embora imponha desafios de granularidade na recuperação. No futuro, planeja-se refinar o *chunking* com base em delimitadores semânticos.

3.2.3 Vetorização

Para cada documento de texto, extraiu-se uma representação numérica por meio de um modelo de aprendizado previamente treinado. Essas representações foram organizadas em uma estrutura de indexação local otimizada para busca por similaridade, de modo a permitir identificar rapidamente documentos próximos no espaço de representações. O sistema de indexação está dimensionado para inserir e consultar centenas de documentos de forma eficiente, garantindo respostas ágeis às consultas de similaridade.

3.2.4 Indexação Vetorial

Após a vetorização dos documentos, os vetores gerados são armazenados em uma estrutura indexada otimizada para buscas por similaridade semântica. O objetivo principal dessa etapa é permitir que, a partir de uma consulta em linguagem natural, o sistema possa recuperar rapidamente os documentos mais semanticamente relacionados, baseando-se na proximidade dos vetores no espaço latente.

O índice utilizado neste trabalho foi configurado para funcionar de forma local, em memória, com persistência em disco. Essa estrutura suporta operações rápidas de inserção e recuperação, mesmo em coleções crescentes de documentos. O método de busca adotado é baseado em distância vetorial, retornando os k vetores mais próximos de uma consulta embutida.

Cada vetor é inserido no índice por meio de lotes, mantendo a consistência entre os identificadores dos documentos e os vetores armazenados. A indexação também preserva metadados relevantes, como o nome do arquivo original e a posição no corpus, permitindo a rastreabilidade dos trechos recuperados. O índice vetorial resultante pode ser salvo em disco e recarregado em execuções futuras, garantindo reutilização e escalabilidade da arquitetura.

3.3 CONSIDERAÇÕES SOBRE O CAPÍTULO

4 ANÁLISE DOS RESULTADOS PARCIAIS

Este capítulo apresenta os resultados preliminares dos testes realizados com o pipeline em desenvolvimento. Os experimentos focaram no desenvolvimento do código necessário ao pipeline e na validação de sua eficácia. A metodologia empregada para a avaliação é descrita na Seção 4.1, enquanto a Seção 4.2 apresenta e analisa os resultados obtidos. Por último, a Seção 4.3 apresenta as considerações sobre os resultados.

4.1 METODOLOGIA DE AVALIAÇÃO

A qualidade das respostas do LLM depende diretamente da relevância dos documentos recuperados do banco vetorial. Portanto, considerando a fase inicial de desenvolvimento do *pipeline*, os experimentos se concentraram em medir a qualidade da recuperação dos arquivos.

Os experimentos realizados na fase atual do trabalho foram feitos com base apenas nos documentos obtidos da seção de "banana" do portal EPAGRI – 73 arquivos ao todo. Para os *embeddings*, considerou-se cada documento como um *chunk*, pois o tamanho médio dos arquivos era de aproximadamente 4300 caracteres (verificado com um script Python).

Para avaliar a acurácia do pipeline indexação e recuperação de artigos, optou-se por utilizar as métricas de precisão e cobertura, conforme definidas em Manning, Raghavan e Schütze (2009). Considera-se TP o número de artigos relevantes recuperados (positivos verdadeiros), FP o número de artigos recuperados não relevantes (falsos positivos) e FN o número de artigos relevantes que não foram recuperados (falsos negativos). A métrica de precisão é dada por

$$\text{Precisão} = \frac{TP}{TP+FP},$$

e a métrica de cobertura (recall) é dada por

$$\text{Cobertura} = \frac{TP}{TP+FN}.$$

Nesse experimento, TP corresponde ao número de artigos relevantes retornados pelo sistema, FP aos artigos retornados mas não pertinentes, e FN aos artigos pertinentes que ficaram de fora dos resultados.

Foram realizados testes usando 3 *strings* simples (semelhante a uma busca por palavra-chave), escolhidas com a ajuda de um especialista na área. Para cada uma das *strings*, verificou-se através de um script Python em quantos documentos ela aparecem ao menos uma vez:

- "sigatoka": 3 ocorrências ao todo. Aparece em contextos extremamente específicos.
- "moko da bananeira": 2 ocorrências ao todo. Não é tão específica, o termo "da bananeira" gera muitas outras similaridades.
- "cavendish": 15 ocorrências ao todo. Termo mais genérico (uma variedade de banana), aparece em diversos documentos.

Nas três consultas, utilizou-se $k = 10$ (para observar em que posição o RAG ranqueava os arquivos), sendo que k é o número de arquivos passados como alvo para o indexador recuperar.

4.2 RESULTADOS

Os resultados para os experimentos foram os seguintes:

- Para a *string* “sigatoka”, havia 3 documentos relevantes no universo total e todos foram recuperados entre os 10 resultados retornados, resultando em precisão de 0,3 (3/10) e cobertura de 1,0 (3/3). Inclusive, os 3 resultados eram os 3 primeiros listados.
- Para a *string* “moko da bananeira”, existiam 2 documentos relevantes no universo e ambos foram recuperados entre os 10 resultados, resultando em precisão de 0,2 (2/10) e cobertura de 1,0 (2/2).
- Para a *string* “cavendish”, dos 10 resultados recuperados 6 eram relevantes, mas o universo continha 15 documentos relevantes, o que gera precisão de 0,6 (6/10) e cobertura de aproximadamente 0,4 (6/15).

4.2.1 Análise Qualitativa dos Resultados

Esses achados mostram que, embora consultas específicas (“sigatoka” e “moko da bananeira”) alcancem cobertura máxima ao recuperar todos os relevantes, a precisão pode ser baixa se o número de resultados recuperado for muito alto. Isso ocorre porque, em cenários com poucos documentos relevantes, a maioria dos itens recuperados tende a não ser útil. Por outro lado, uma consulta mais genérica (“cavendish”) pode apresentar precisão relativamente maior, mas cobertura limitada ao não alcançar todos os documentos relevantes disponíveis.

Logo, equilibrar precisão e cobertura é um *trade-off* entre recuperar poucos documentos (bom se o contexto fornecido pelo usuário for adequado) ou muitos documentos (bom se o usuário fornecer um conteúdo parcialmente relacionado), que tem o potencial de ser resolvido com o uso de um LLM. A literatura menciona ferramentas para adaptar o ranqueamento dos resultados ou deixar o LLM decidir quais resultados são relevantes ou não: a partir do ponto em que os resultados ficam muito distantes semanticamente dos primeiros resultados, o modelo altera o tamanho da janela de recuperação - o valor k.

As primeiras impressões indicam que o sistema retorna documentos relevantes em aproximadamente 12 segundos. Métricas mais precisas de desempenho serão obtidas com a ampliação da base de documentos, a formalização de uma lista de consultas-teste e a validação com auxílio de especialistas da área agrícola.

4.3 CONSIDERAÇÕES SOBRE OS RESULTADOS

Até o momento, a coleta cobre um dos principais menus de navegação (a parte do portal sobre banana), clicando em todos os botões daquele grupo e salvando o HTML completo de cada URL. A arquitetura não padronizada do site — estruturas de DIVs e classes diferentes em cada página — exigiu rotinas específicas de localização de elementos, o que aumentou o esforço de desenvolvimento. Além disso, muitas imagens presentes não contêm descrições no HTML; por isso, um especialista em agronomia ajudou a classificar quais imagens são relevantes. A proposta futura é gerar descrições textuais dessas imagens (legendas sintéticas) e incorporá-las ao corpo dos documentos para enriquecer o contexto semântico.

Em relação aos resultados de precisão e cobertura na recuperação das *strings* testadas, observou-se que os principais falsos positivos eram elementos semanticamente próximos, algo positivo em relação à recuperação das informações.

Outra consideração importante é o impacto do tamanho de cada arquivo texto nos *embeddings*: os trabalhos relacionados comumente dividem os mesmos em *chunks* de tamanho padronizado, compatível com o *embedder* utilizado. Durante a realização da etapa inicial desse trabalho, teve-se a impressão de que cada arquivo texto obtido da limpeza da coleta dos dados já era pequeno o suficiente para ser considerado um *chunk*, mas as Figuras 3 e 4 mostram evidências de que na próxima etapa do trabalho isso deve ser alterado.

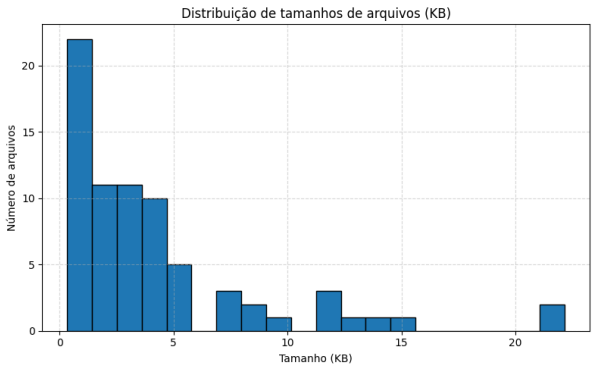

Figura 3 – Distribuição do tamanho dos documentos utilizados. Fonte: O autor.

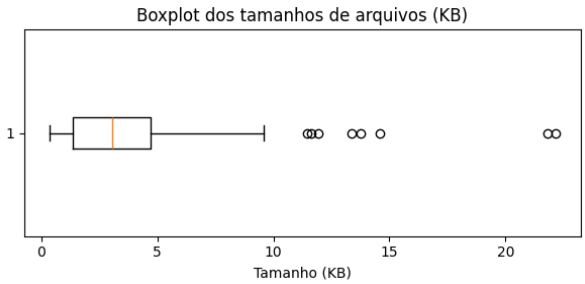

Figura 4 – Boxplot do tamanho dos documentos utilizados. Fonte: O autor.

Na média os arquivos usados excedem o tamanho máximo do modelo utilizado como *embedder* (512 *tokens*). Lewis et al. (2020) usam blocos de texto relativamente pequenos (em

torno de 100 palavras) para não exceder o limite de tokens. Ainda não é claro o impacto que isso teve nos resultados, mas a expectativa para o a sequência do trabalho é que a divisão do texto emw pedaços semanticamente menores aumente ainda mais a capacidade do modelo de encontrar trechos relevantes.

5 CONSIDERAÇÕES PARCIAIS

Até o momento, o *pipeline* consolidou a coleta automatizada de parte do portal EpagriTec, o pré-processamento básico, a vetorização e a subsequente indexação vetorial. Esses avanços validam a viabilidade técnica da proposta e fornecem uma base sólida para prosseguir às etapas de fragmentação semântica (*chunking* refinado), incorporação de descrições de imagens e, possivelmente, a geração de respostas por um LLM integrado.

Em testes preliminares, a indexação demonstrou bom desempenho para inserção e busca em uma coleção com dezenas de documentos, apresentando latência compatível com aplicações interativas. A escolha pela FAISS justifica-se pelo seu desempenho consolidado, ampla adoção na literatura e suporte a diversas estratégias de indexação — como flat, IVF ou HNSW — que podem ser exploradas em etapas futuras do projeto.

Os resultados parciais revelam que a arquitetura pode escalar para centenas de páginas, o que demanda leves alterações no processo inicial de coleta de dados; e que a busca vetorial atua com latência compatível a aplicações em tempo real, porém o tamanho dos *chunks* deverá ser refinada para aumentar a precisão.

Próximos passos incluem: Chunking semântico automático, dividindo documentos em fragmentos de tamanho ideal; geração de legendas para imagens selecionadas e inserção no texto, ampliando o contexto; uma possível integração com LLM (RAG completo), avaliando a qualidade das respostas geradas; avaliação quantitativa e qualitativa com métricas padrão de recuperação (precisão, recall, NDCG) e estudos de caso com profissionais da área. Além disso, a etapa de pré-processamento dos documentos será refinada para excluir os menus (algo já mencionado nos trabalhos relacionados, mas que até o momento não se conseguiu fazer de forma automática devido à estrutura do portal EpagriTEC).

Como validação adicional, se a etapa de geração de respostas pelo LLM for feita com sucesso, será possível aplicar métricas para determinar a acurácia do *pipeline* a partir dos textos gerados pelo LLM (em comparação com as respostas esperadas).

Assim, o trabalho avança rumo a um protótipo funcional capaz de oferecer suporte à tomada de decisão em cultivos agrícolas de Santa Catarina, combinando dados atualizados via scraping e poder de geração de LLMs.

5.1 CRONOGRAMA DAS PRÓXIMAS ETAPAS

A Tabela 2 apresenta o cronograma que será seguido na próxima etapa do trabalho, seguindo as seguintes etapas:

- Etapa 1: Aperfeiçoar o pipeline já proposto, integrando os elementos ainda não explorados e já mencionados nessa Seção.
- Etapa 2: Experimentação do pipeline e aplicação de métricas para avaliação mais precisa dos resultados.

- Etapa 3: Finalização da monografia do TCC.

Etapas	TCC2 – 2025/2					
	JUL	AGO	SET	OUT	NOV	DEZ
1	X	X	X	X		
2		X	X	X	X	
3		X	X	X	X	X

Tabela 2 – Cronograma de Tarefas – TCC2 (2025/2)

REFERÊNCIAS

AKBAR, Nur Arifin. Are re-ranking in retrieval-augmented generation methods impactful for small agriculture qa datasets? a small experiment. In: **BIO Web of Conferences, 5th International Conference on Smart and Innovative Agriculture (ICoSIA 2024)**. [S.l.: s.n.], 2025. v. 167, p. 01001. Citado 2 vezes nas páginas 21 e 23.

ANNEPAKA, Yadagiri; PAKRAY, Partha. Large language models: a survey of their development, capabilities, and applications. **Knowledge and Information Systems**, v. 67, n. 3, p. 2967–3022, March 2025. ISSN 0219-3116. Disponível em: <https://doi.org/10.1007/s10115-024-02310-4>. Citado na página 10.

BASSOI, Luís Henrique et al. Agricultura de precisão e agricultura digital. **TECCOGS: Revista Digital de Tecnologias Cognitivas**, v. 20, p. 17–36, 2019. Disponível em: <http://dx.doi.org/10.23925/1984-3585.2019i20p17-36>. Citado na página 9.

BRASIL, Ministério da Agricultura e Pecuária. **Crescimento da economia brasileira é impulsionado pela alta de 15% da agropecuária em 2023**. 2024. Acesso em: 18 mar. 2025. Disponível em: <https://www.gov.br/agricultura/pt-br/assuntos/noticias/crescimento-da-economia-brasileira-e-impulsionado-pela-alta-de-15-da-agropecuaria-em-2023>. Citado na página 9.

CAMACHO-COLLADOS, Jose; PILEHVAR, Mohammad Taher. On the role of text preprocessing in neural network architectures: An evaluation study on text categorization and sentiment analysis. Association for Computational Linguistics, Brussels, Belgium, p. 40–46, nov. 2018. Disponível em: <https://aclanthology.org/W18-5406/>. Citado na página 15.

DAS, Utpal et al. Precision farming a promising technology in horticulture: A review. **International Journal of Pure & Applied Bioscience**, v. 6, n. 1, p. 1596–1606, 2018. Disponível em: <http://dx.doi.org/10.18782/2320-7051.3088>. Citado na página 9.

ELOV, B. B.; KHAMROEVA, Sh. M.; XUSAINOVA, Z. Y. The pipeline processing of nlp. In: **E3S Web of Conferences**. [S.l.: s.n.], 2023. v. 413, p. 03011. Citado na página 15.

FERNÁNDEZ-PICHEL, Marcos et al. An unsupervised perplexity-based method for boilerplate removal. **Natural Language Engineering**, v. 30, n. 1, p. 132–149, 2024. Citado na página 15.

GAO, Yunfan et al. Retrieval-augmented generation for large language models: A survey. **arXiv preprint arXiv:2312.10997**, 2023. Citado na página 17.

GLUKHIKH, Igor N.; CHERNYSHEVA, Tatiana Y.; SHENTSOV, Yaroslav A. Decision support in a smart greenhouse using large language model with retrieval augmented generation. In: **Proc. SPIE 13217 (DTIEE 2024)**. [S.l.: s.n.], 2024. Citado 3 vezes nas páginas 19, 21 e 23.

IBGE, Instituto Brasileiro de Geografia e Estatística. **PIB cresce 2,9% em 2023 e fecha o ano em R$ 10,9 trilhões**. 2024. Acesso em: 18 mar. 2025. Disponível em: <https://agenciadenoticias.ibge.gov.br/agencia-sala-de-imprensa/2013-agencia-de-noticias/releases/39303-pib-cresce-2-9-em-2023-e-fecha-o-ano-em-r-10-9-trilhoes>. Citado na página 9.

JIANG, Jingchi et al. Knowledge assimilation: Implementing knowledge-guided agricultural large language model. **Knowledge-Based Systems**, p. 113197, 2025. Citado 3 vezes nas páginas 18, 21 e 23.

KAGERMANN, Henning; WAHLSTER, Wolfgang; HELBIG, Johannes. **Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0: Final Report of the Industrie 4.0 Working Group**. [S.l.], 2013. Disponível em: <https://en.acatech.de/publication/recommendations-for-implementing-the-strategic-initiative-industrie-4-0-final-report-of-the-industrie-4-0-w>. Citado na página 9.

KHDER, Moaiad Ahmad. Web scraping or web crawling: State of art, techniques, approaches and application. **International Journal of Advanced Soft Computing and Applications**, v. 13, n. 3, p. 145–168, November 2021. ISSN 2710-1274. Accessed: 2025-03-19. Disponível em: <https://www.researchgate.net/publication/357401723_Web_Scraping_or_Web_Crawling_State_of_Art_Techniques_Approaches_and_Application>. Citado na página 13.

LAWSON, Richard. **Web Scraping with Python**. Birmingham: Packt Publishing Ltd, 2015. Citado 2 vezes nas páginas 10 e 13.

LEWIS, Patrick et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. **arXiv preprint arXiv:2005.11401**, 2020. Citado na página 30.

MA, Hao et al. Sdd-lawllm: Advancing intelligent legal systems through synthetic data-driven fine-tuning of large language models. **Electronics**, v. 14, n. 4, p. 742, 2025. Citado 3 vezes nas páginas 20, 21 e 23.

MANNING, Christopher D.; RAGHAVAN, Prabhakar; SCHüTZE, Hinrich. **Introduction to Information Retrieval**. Cambridge, UK: Cambridge University Press, 2009. Online edition draft April 1, 2009. Disponível em: <https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>. Citado na página 28.

MINAEE, Shervin; MIKOLOV, Tomas et al. Large language models: A survey. **Artificial Intelligence Review**, 2024. Preprint (ainda não revisado por pares). Acessado em: 9 de junho de 2025. Disponível em: <https://doi.org/10.48550/arXiv.2402.06196>. Citado na página 13.

MITCHELL, Ryan. **Web Scraping with Python: Collecting Data from the Modern Web**. 2nd. ed. Sebastopol, CA: O'Reilly Media, 2018. ISBN 978-1491985571. Citado na página 13.

MURTAZA, Syed Shariyar et al. Implementing retrieval augmented generation technique on unstructured and structured data sources in a call center of a large financial institution. In: **Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL) – Industry Track**. [S.l.: s.n.], 2025. p. 598–606. Citado 2 vezes nas páginas 16 e 17.

OLIVEIRA, Rosana Cavalcante de; SILVA, Rogério Diogne de Souza e. Artificial intelligence in agriculture: Benefits, challenges, and trends. **Applied Sciences**, v. 13, n. 13, 2023. ISSN 2076-3417. Disponível em: <https://www.mdpi.com/2076-3417/13/13/7405>. Citado na página 9.

PAN, James Jie; WANG, Jianguo; LI, Guoliang. Vector database management techniques and systems. In: **SIGMOD 2024 Companion**. [S.l.: s.n.], 2024. p. 529–540. Citado na página 16.

PANOUTSOPOULOS, Hercules et al. Investigating the effect of different fine-tuning configuration scenarios on agricultural term extraction using bert. **Computers and Electronics in Agriculture**, v. 225, p. 109268, 2024. Citado 3 vezes nas páginas 18, 21 e 23.

PERSSON, Emil. **Evaluating tools and techniques for web scraping**. Dissertação (Degree Project in Computer Science and Engineering, Second Cycle) — KTH Royal Institute of Technology, Stockholm, Sweden, 2019. Acessado em: 12 de junho de 2025. Disponível em: <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1345238>. Citado na página 13.

RAMYA, C; SHREEDHARA, KS; KAVITHA, G. Preprocessing: A prerequisite for discovering patterns in wum process. **Computing Research Repository - CORR**, 04 2011. Citado na página 14.

REIMERS, Nils; GUREVYCH, Iryna. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: **Proceedings of EMNLP-IJCNLP 2019**. Hong Kong, China: ACL, 2019. p. 3982–3992. Citado na página 16.

REZAYI, Saed et al. AgriBERT: Knowledge-infused agricultural language models for matching food and nutrition. In: **Proc. IJCAI 2022 (AI for Good)**. [S.l.]: IJCAI Organization, 2022. p. 5150–5156. Citado 3 vezes nas páginas 18, 21 e 23.

SAMUEL, Dinesh Jackson et al. **AgroLLM: Connecting Farmers and Agricultural Practices through Large Language Models for Enhanced Knowledge Transfer and Practical Application**. 2025. Disponível em: <https://arxiv.org/abs/2503.04788>. Citado na página 20.

SHEHU, Mustapha et al. A demystified overview of data scraping. **International Journal of Data Science and Advanced Analytics**, v. 6, n. 6, 2024. Citado 2 vezes nas páginas 5 e 14.

SOHN, Jiwoong et al. Rationale-guided retrieval augmented generation for medical question answering. In: **Proc. NAACL 2025**. [S.l.: s.n.], 2025. p. 12739–12753. Citado 3 vezes nas páginas 19, 21 e 23.

TARANNUM, Tasnuva. **Cleaning of Web Scraped Data with Python**. Tese (Doutorado) — Brac University, apr 2019. Dissertação (B.Sc. CSE). Disponível em: <http://hdl.handle.net/10361/12354>. Citado na página 14.

UPADHYE, Akshata. A comprehensive survey of text data cleaning techniques: Challenges, methods, and best practices. **Journal of Scientific and Engineering Research**, v. 7, n. 8, p. 205–210, 2020. Citado na página 15.

VAGHEFI, Saeid Ashraf et al. Chatclimate: Grounding conversational ai in climate science. **Communications Earth & Environment**, v. 4, p. 480, 2023. Citado 3 vezes nas páginas 19, 21 e 23.

WEERASINGHE, Manushi. Enhancing web scraping with artificial intelligence: A review. In: KOTELAWALA DEFENCE UNIVERSITY. **Proceedings of the 4th Research Symposium of the Faculty of Computing**. 2024. Acessado em: 8 de junho de 2025. Disponível em: <https://www.researchgate.net/publication/379024314_Enhancing_Web_Scraping_with_Artificial_Intelligence_A_Review>. Citado na página 14.

XIONG, Jie et al. Enhancing plant protection knowledge with large language models: A fine-tuned qa system using lora. **Applied Sciences**, v. 15, n. 7, p. 3850, 2025. Citado 3 vezes nas páginas 19, 21 e 23.

YANG, Shuting et al. Shizishangpt: An agricultural large language model integrating tools and resources. **arXiv preprint arXiv:2409.13537**, 2024. Citado na página 20.

ZHANG, Zhihao et al. Beefbot: Harnessing advanced LLM and RAG techniques for providing scientific and technology solutions to beef producers. In: **Proceedings of the 31st International Conference on Computational Linguistics: System Demonstrations (COLING Demos 2025)**. Abu Dhabi, UAE: Association for Computational Linguistics, 2025. p. 54–62. Disponível em: <https://aclanthology.org/2025.coling-demos.7/>. Citado 4 vezes nas páginas 18, 21, 22 e 23.

ZHAO, Wayne Xin et al. A survey of large language models. **arXiv preprint arXiv:2303.18223**, 2023. Preprint (ainda não revisado por pares). Acessado em: 8 de junho de 2025. Disponível em: <https://arxiv.org/abs/2303.18223>. Citado na página 13.